

More Classes and Objects

CSCI 1101

2024-04-12

Modeling a Ball

Today we'll model the motion of a ball in 2-dimensions.

We'll start by keeping track of the following information:

position: where the ball is located,

velocity: how the ball is moving.

We'll represent this information using:

position: `tuple [int, int]`

velocity: `tuple [int, int]`

A Ball Class

Let's start writing a Ball **class**:

```
class Ball :
```

We want each Ball **object** to have a position and velocity.

So we'll add an **initializer method** that takes this information as arguments and stores it in a ball's **attributes**:

```
def __init__(self , init_pos , init_vel) :  
    self.position = init_pos  
    self.velocity = init_vel
```

Representing a Ball as Text

Python reserves the following special method names:

- “`__str__`” for converting an object to a string using the `str` function,
- “`__repr__`” for displaying an object in the interactive interpreter.

It will be convenient to have these representations for balls,

so we add the following methods to the `Ball` class:

```
def __str__(self) :  
    return f"Ball p:{self.position} v:{self.velocity}"  
  
def __repr__(self) :  
    return str(self)
```

Interacting with a Ball

Rather than directly modifying an object's attributes, it is good practice to interact with it using an **interface**.

So we'll add some **methods** to our Ball class:

```
def get_pos(self) :  
    return self.position
```

```
def get_vel(self) :  
    return self.velocity
```

A Ball That Moves

Finally, we want our ball to *move*. One strategy is tell the ball where to go.

Another strategy is to let the ball figure out where to go using *physics*.

We add the following method to our Ball class:

```
def move(self , time) :  
    # get the current position:  
    (p_x , p_y) = self.position  
    # get the current velocity:  
    (v_x , v_y) = self.velocity  
    # update the current position:  
    self.position = (p_x+(v_x*time) , p_y+(v_y*time))
```

Now if we tell our ball how much time has passed, it will know where to go.

But watching a ball move this way is pretty boring.

Turtle Setup

Let's do some animation with `turtle` instead:

```
import random, time, turtle
```

```
dot_size = 10
```

```
timestep = 1/60
```

```
def init_turtle() :
```

```
    # don't show the turtle icon:
```

```
    turtle.hideturtle()
```

```
    # default turtle state is not drawing:
```

```
    turtle.penup()
```

```
    # don't render to screen unless instructed:
```

```
    turtle.tracer(0 , 0)
```

Animating a Ball

We write a function to draw a ball at a given position:

```
def draw_ball(ball) :  
    (x , y) = ball.get_pos()  
    turtle.goto(x , y)  
    turtle.dot(dot_size)
```

And one to animate the ball as time passes:

```
def run_ball(ball) :  
    init_turtle()  
    while True :                # an event loop  
        turtle.clear()          # erase the old scene  
        draw_ball(ball)  
        ball.move(timestep)  
        turtle.update()         # paint the new scene  
        time.sleep(timestep)
```


Multi-Ball

One ball is okay, but more would be better:

```
# a set of balls:
balls = set()

max_speed = 500

# add a random ball to the set:
def add_ball() :
    x_vel = random.randint(-max_speed , max_speed)
    y_vel = random.randint(-max_speed , max_speed)
    new_ball = Ball((0 , 0) , (x_vel , y_vel))
    balls.add(new_ball)
```

Ball Runner

Now we need a runner for multiple balls:

```
def run_balls() :  
    init_turtle()  
    turtle.onkeypress(add_ball , 'space')  
    turtle.listen()  
    while True :  
        # an event loop  
        turtle.clear()          # erase the old scene  
        for ball in balls :  
            draw_ball(ball)  
            ball.move(timestep)  
        turtle.update()         # paint the new scene  
        time.sleep(timestep)
```

Bouncing Balls

Our balls are getting away from us. Let's try to keep them more contained.

We'll write functions to reflect an ordered pair in each dimension:

```
def reflect_x (pair) :  
    (x , y) = pair  
    return (-x , y)
```

```
def reflect_y (pair) :  
    (x , y) = pair  
    return (x , -y)
```

Bouncing Balls ctd.

We can use these to update our `Ball.move` method to make a ball bounce back when it goes too far:

```
def move(self , time) :  
    (p_x , p_y) = self.position  
    if abs(p_x) >= 250 :  
        self.velocity = reflect_x(self.velocity)  
    if abs(p_y) >= 250 :  
        self.velocity = reflect_y(self.velocity)  
    (v_x , v_y) = self.velocity  
    self.position = (p_x+(v_x*time) , p_y+(v_y*time))
```

Activities

You can try to improve the ball simulation by adding features such as:

- gravity,
- inelastic collisions,
- angular momentum,
- interactions between balls.

To Do

Finish *Homework 10: Object-Oriented Programming* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 3 - Automated Grader (Part 2)* on CodeRunner by 12:00PM next Friday.